

Name: Amma Anjali

Hall No : 2303a52385

Delete a node from a binary search tree.

DescriptionEditorialSolutionsSubmissions

450. Delete Node in a BST

Solved

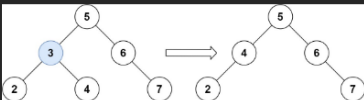
MediumTopicsCompanies

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the **root node reference** (possibly updated) of the BST.

Basically, the deletion can be divided into two stages:

- Search for a node to remove.
- If the node is found, delete the node.

Example 1:



10.6K26569 Online

Code

JavaAuto

```
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) return null;
        if (root.val < key) {
            root.left = deleteNode(root.left, key);
        } else if (root.val > key) {
            root.right = deleteNode(root.right, key);
        } else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;
            else {
                TreeNode succ = findMin(root.right);
                root.val = succ.val;
                root.right = deleteNode(root.right, succ.val);
            }
        }
        return root;
    }
}
```

SavedLn 1, Col 1

TestcaseTest Result

Check if a binary tree is a binary search tree.

DescriptionEditorialSolutionsSubmissions

98. Validate Binary Search Tree

Solved

MediumTopicsCompanies

Given the **root** of a binary tree, determine if it is a **valid binary search tree (BST)**.

A **valid BST** is defined as follows:

- The left **subtree** of a node contains only nodes with keys **strictly less than** the node's key.
- The right subtree of a node contains only nodes with keys **strictly greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:

Code

JavaAuto

```
TreeNode(int val, TreeNode left, TreeNode right) {
    this.val = val;
    this.left = left;
    this.right = right;
}

class Solution {
    public boolean isValidBST(TreeNode root) {
        return helper(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    boolean helper(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false;
        return helper(node.left, min, node.val) && helper(node.right, node.val, max);
    }
}
```

Restored from localUpgrade to Cloud SavingLn 1, Col 1

TestcaseTest Result

Find the kth smallest and kth largest elements in a BST.

Description

Editorial

Solutions

Submissions

230. Kth Smallest Element in a BST

Medium

Given the `root` of a binary search tree, and an integer `k`, return the k^{th} smallest value (1-indexed) of all the values of the nodes in the tree.

Example 1:

```

graph TD
    3((3)) --- 1((1))
    3 --- 4((4))

```

12.9K 228 114 Online

Code

```

16 class Solution {
17     int count=0;
18     int res=0;
19     public int kthSmallest(TreeNode root, int k) {
20         helper(root,k);
21         return res;
22     }
23
24     void helper(TreeNode root,int k){
25         if(root==null)return ;
26         helper(root.left,k);
27         count++;
28         if(count==k){
29             res=root.val;
30             return;
31         }
32         helper(root.right,k);
33     }
34 }

```

Saved Ln 1, Col 1

Testcase Test Result

Find whether the given BST is balanced or not.

Description

Editorial

Solutions

Submissions

110. Balanced Binary Tree

Easy

Given a binary tree, determine if it is **height-balanced**.

Example 1:

```

graph TD
    3((3)) --- 9((9))
    3 --- 20((20))
    20 --- 15((15))
    20 --- 7((7))

```

Input: root = [3,9,20,null,null,15,7]

12.4K 323 130 Online

Code

```

14 *
15 */
16 class Solution {
17     public boolean isBalanced(TreeNode root) {
18         if(root==null){
19             return true;
20         }
21         return helper(root)!=-1;
22     }
23
24     int helper(TreeNode root){
25         if(root==null){
26             return 0;
27         }
28         int left=helper(root.left);
29         if(left==-1){
30             return -1;
31         }
32         int right=helper(root.right);
33         if(right==-1){
34             return -1;
35         }

```

Restored from local Upgrade to Cloud Saving Ln 1, Col 1

Testcase Test Result